B.TECH FINAL YEAR MAJOR PROJECT — PROPOSAL

Scope-Bound Dynamic Consent and Anomaly-Aware Emergency Access Control for Blockchain-Based Electronic Health Record Systems

(MediVault AI)

EXTENDED PROPOSAL — WITH ZERO-KNOWLEDGE STRETCH GOAL

Department of Data Engineering
MVGR College of Engineering (Autonomous), Vizianagaram
R23 Regulation — VI Semester
Submitted by: Karna (23331A4722) and Team — Team of 4

Abstract

Electronic Health Record (EHR) systems increasingly need to balance two competing demands: giving patients meaningful control over who accesses their medical data, and ensuring clinicians can access critical information during emergencies without unnecessary delay. Existing blockchain-based EHR systems typically address these needs with coarse, binary access control — a doctor is either granted or denied access — and treat emergency access as an all-or-nothing override with minimal accountability.

This project proposes a blockchain-based EHR framework built around three core contributions. First, **scope-bound dynamic consent**, where access permissions are enforced at the smart-contract level and are bound to a specific record type, duration, and stated purpose, rather than granting blanket access to a patient's full record. Second, **anomaly-aware access monitoring**, where a lightweight machine learning model flags requests that deviate from a clinician's historical access pattern, surfacing this to the patient before approval rather than simply logging it after the fact. Third, **tiered emergency ("break-glass") access with mandatory post-hoc reconciliation**, where emergency access is graduated based on verifiable signals and every emergency access event is later presented to the patient for review, with disputes recorded immutably alongside the original event.

The framework additionally explores a zero-knowledge proof mechanism for privacy-preserving emergency verification as a stretch goal beyond the core contributions.

The system is implemented using Ethereum-compatible smart contracts (developed and tested locally using Ganache) for permission enforcement and audit logging, off-chain encrypted storage (IPFS) for medical records, and a scikit-learn-based anomaly detection model trained on synthetic access-pattern data. The framework is evaluated for security guarantees, transaction/performance overhead, and consent transparency.

Problem Statement

Existing Electronic Health Record systems, including recent blockchain-based proposals, exhibit recurring limitations:

1. **Consent is coarse-grained.** Access is typically modeled as a binary permission applied to a patient's entire record set, rather than being scoped to specific record types, time windows, or stated clinical purposes.
2. **Emergency access lacks graduated accountability.** Break-glass mechanisms in most systems grant full, unrestricted access once triggered, with audit logging occurring only after the fact.
3. **Access anomalies are not surfaced to the data owner.** Patients have no practical way to identify unusual or potentially inappropriate access patterns without manually auditing every log entry.
4. **Patients disengage from consent decisions over time.** Frequent, low-context consent requests lead to notification fatigue, causing patients to approve requests without meaningful review — a known failure mode that undermines the purpose of dynamic consent systems.
5. **Deletion rights conflict with blockchain immutability.** Data protection principles that grant patients a right to withdraw or delete records are structurally difficult to reconcile with an append-only ledger, and most existing systems do not address this tension explicitly.

This project addresses these gaps directly, rather than proposing another general-purpose blockchain EHR architecture.

Objectives

1. Design and implement a smart-contract layer where access grants are bound to record type, duration, and purpose, and are enforced at the contract level rather than the application layer.
2. Develop an anomaly-detection component that scores each access request against a clinician's historical access pattern and flags deviations for patient review prior to approval.
3. Implement a tiered break-glass access mechanism where the scope of emergency access depends on the strength of identity/emergency verification available at request time.
4. Build a patient-facing reconciliation workflow allowing formal, immutably-logged review and dispute of emergency access events.
5. Design a consent-batching mechanism that groups related, low-risk access requests to reduce patient notification fatigue without weakening oversight of high-risk requests.
6. Evaluate the system for security, performance overhead, and usability of the consent and audit interfaces.

Research Questions

1. Can access permissions be enforced at the smart-contract level with fine enough granularity to meaningfully reduce over-exposure of patient data, without excessive performance cost?
2. Can a lightweight anomaly-detection model, trained on access-pattern data, reliably flag out-of-pattern clinician requests as a meaningful pre-approval signal for patients?
3. Does a tiered, verification-dependent break-glass model provide a better security/availability trade-off than all-or-nothing emergency access?
4. Does a mandatory post-hoc reconciliation workflow measurably improve patient trust and perceived accountability compared to audit-log-only systems?
5. Does batching related low-risk consent requests reduce patient notification fatigue without reducing the quality of oversight on high-risk requests?

Scope

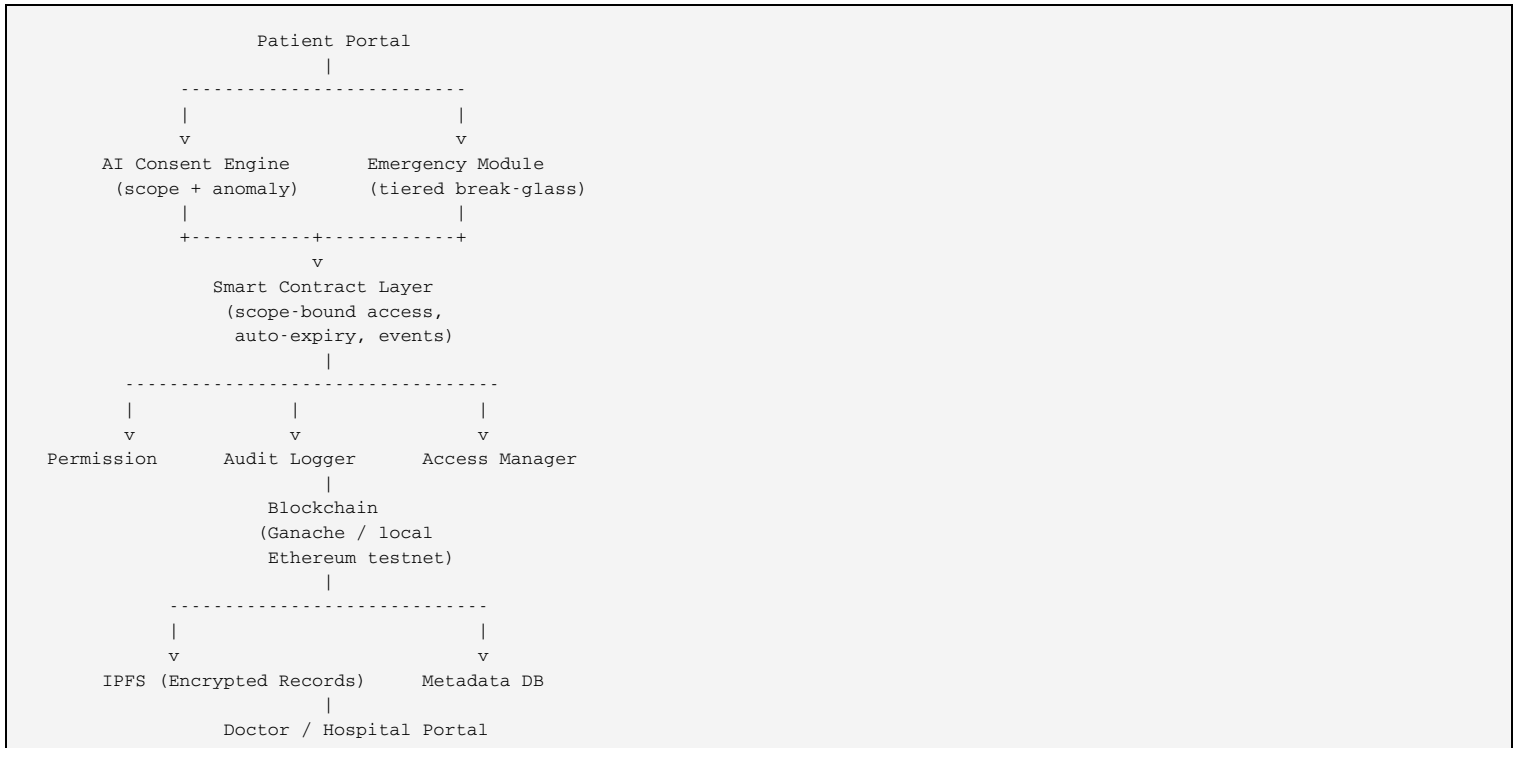
In Scope (Built and Evaluated)

- Patient and doctor portals (registration, record upload, request/grant/revoke access)
- Solidity smart contracts for scope-bound access grants and break-glass logic with auto-expiry, developed and tested on a local Ganache network
- IPFS-based encrypted record storage with on-chain hash anchoring
- scikit-learn anomaly-detection model trained on synthetic access-pattern data
- Consent-batching logic for low-risk, related access requests
- Audit dashboard with patient-facing dispute/reconciliation flow
- Design-level treatment of the deletion-versus-immutability tension (tombstone-record approach)

Out of Scope (Future Work)

- Multi-factor authentication, SMS/email notification integration
- QR-based sharing, hospital-system interoperability (HL7/FHIR integration)
- Production-grade consortium blockchain (Hyperledger Fabric) deployment
- Real clinical data or IRB-approved clinical trials
- Full production zero-knowledge proof integration (explored only as a stretch goal / design-level contribution)

Proposed Architecture



Core Modules

Module 1 — Patient Portal

Registration, record upload, viewing history, granting and revoking scoped access, receiving notifications (individually or as batched groups), and reviewing the audit and reconciliation log.

Module 2 — Doctor Portal

Requesting scoped access with a stated purpose, viewing authorized records, uploading prescriptions or diagnoses, and requesting tiered emergency access.

Module 3 — Scope-Bound Consent Engine

Access grants are not simple approve/deny decisions. Each grant is bound to a specific record type, time window, and stated purpose, and is enforced at the smart-contract level — a request outside the granted scope is rejected by the contract itself, not merely flagged by the application layer.

Module 4 — Anomaly-Aware Access Scoring

A scikit-learn model scores each access request against the requesting clinician's historical access pattern (specialty, prior request types, frequency, hospital affiliation). Requests that deviate significantly from established patterns are flagged for closer patient review before approval, rather than only being logged after the fact.

Module 5 — Consent Batching

Related, low-risk requests — for example, multiple clinicians on the same care team requesting the same routine follow-up record — are grouped into a single consent decision for the patient, while individually flagged high-risk or anomalous requests are still escalated separately. This reduces notification fatigue without reducing oversight where it matters most.

Module 6 — Tiered Emergency Access (Break-Glass)

Emergency access is graduated rather than all-or-nothing:

```
Hospital identity verified only
        -> Partial access (allergies, medications, blood type)

Hospital identity + declared emergency code verified
        -> Full access, time-limited, auto-revoked

All emergency access events are logged immediately and the patient is notified as soon as possible.
```

Module 7 — Post-Emergency Reconciliation

Every emergency access event is presented to the patient after the fact for formal review. The patient may raise a dispute, which is itself recorded immutably alongside the original access event — creating an accountability trail that cannot be quietly altered by either party.

Module 8 — Blockchain and Storage Layer

The blockchain stores record hashes, signatures, access permissions, audit logs, emergency events, and consent history. Large files (imaging, PDFs) are encrypted and stored off-chain on IPFS, with only their SHA-256 hash anchored on-chain.

Module 9 — Audit Dashboard

A patient-facing dashboard showing who accessed which records, when, why, for how long, and whether the access was routine or emergency — including batched-consent groupings and any open or resolved disputes.

Module 10 — Zero-Knowledge Emergency Verification (Stretch Goal) STRETCH GOAL

As an additional, higher-risk contribution, the project explores a zero-knowledge proof scheme that allows a hospital to prove possession of a valid emergency-authorization credential without revealing the specific credential or patient context until after the access event is logged. This adds a privacy-preserving verification layer on top of the tiered break-glass model in Module 6.

Given its complexity relative to the team's current experience with smart contract development, this component is treated as a stretch goal: the team will implement a simplified proof-of-concept or, at minimum, produce a fully reasoned design, and will not allow it to block delivery of the core system described above.

Novel Contributions

Most existing blockchain-based EHR proposals provide only basic, binary permission management and treat emergency access as an unrestricted override. This project's contributions are:

- **Scope-bound, contract-enforced dynamic consent** — access is limited by record type, duration, and purpose, and this is enforced at the smart-contract layer rather than only in application logic.
- **Anomaly-aware access scoring** — a lightweight machine learning model that gives the patient a meaningful, pre-approval signal about whether a request fits the requesting clinician's established pattern of access.
- **Tiered break-glass access with mandatory post-hoc reconciliation** — emergency access is graduated by verification strength, and every emergency event is subject to formal patient review and dispute, recorded immutably.
- **Consent batching to mitigate notification fatigue** — related, low-risk requests are grouped for a single patient decision, addressing a documented failure mode of dynamic consent systems without weakening scrutiny of high-risk requests.
- **Explicit treatment of the deletion-versus-immutability tension** — rather than ignoring the conflict between data-protection deletion rights and blockchain immutability, the design incorporates a tombstone-record approach and discusses this trade-off directly.
- **Zero-knowledge emergency verification (stretch goal)** — a privacy-preserving proof mechanism allowing hospitals to verify emergency authorization without prematurely revealing patient context, explored as an additional contribution beyond the core system.

Technology Stack

LAYER	TECHNOLOGY
Frontend	React.js
Backend	Node.js / Express (API layer connecting frontend to contracts)
Blockchain	Solidity smart contracts, developed and tested locally with Ganache
Contract Tooling	Hardhat / Truffle, ethers.js
Database	PostgreSQL (metadata)
File Storage	IPFS (encrypted medical records)
Machine Learning	Python, scikit-learn (anomaly detection on synthetic access-pattern data)
Authentication	JWT-based session authentication
Encryption	AES-256 (records), SHA-256 (hash anchoring)

Team and Role Distribution

The project is undertaken by a team of four, with responsibilities divided along natural architectural seams so that the blockchain and machine learning components can be developed in parallel from the outset, while backend and frontend work proceeds once contract interfaces are agreed upon.

ROLE	RESPONSIBILITY
Blockchain / Smart Contracts	Solidity contracts for scope-bound grant/revoke, break-glass and auto-expiry logic, on-chain event/audit logging, Ganache-based local testing.
Backend & Storage	API layer bridging frontend and contracts, IPFS integration, AES-256 encryption, hash anchoring, database design.
Frontend	Patient and doctor portals, audit dashboard, and the reconciliation/dispute interface.
AI / ML and Evaluation	Synthetic dataset generation, anomaly-detection model, integration as a scoring API, and the overall security/performance evaluation write-up.

> The smart contract's public functions and event schema will be finalized as a shared interface early in the project, even before all internal logic is complete, so that backend and frontend development are not blocked waiting on the blockchain module.

Expected Deliverables

- Web application with patient and doctor portals
- Solidity smart contracts deployed and tested on a local Ganache network
- Scope-bound access control and tiered break-glass logic with auto-expiry
- Anomaly-detection scoring engine and consent-batching logic
- Encrypted, IPFS-based medical record storage with on-chain hash anchoring
- Audit dashboard with post-emergency reconciliation and dispute workflow
- Performance and security evaluation
- Final project report documenting design, implementation, and results
- Proof-of-concept or design document for zero-knowledge emergency verification (stretch goal)

Indicative Timeline

PHASE	FOCUS
Weeks 1–2	Literature review, finalize scope with guide, environment setup (Ganache, Hardhat/Truffle, React, Node.js), shared contract interface draft
Weeks 3–4	Core smart contracts: scope-bound grant/revoke, unit testing
Weeks 5–6	Patient portal: authentication, upload, encryption, hash anchoring
Weeks 7–8	Doctor portal: access requests, viewing granted records
Weeks 9–10	Tiered break-glass module, auto-revocation, reconciliation workflow
Weeks 11–12	Anomaly-detection model: synthetic dataset, training, scoring API, consent-batching logic
Week 13	Audit dashboard and integration of all modules
Week 14	Security and performance evaluation
Weeks 15–16	Report writing, diagrams, viva preparation, buffer

This timeline should be aligned against the department's actual review checkpoints and academic calendar before finalization.
